

Programming_

照会

```
const BASE = 'http://localhost:8080/api'

export async function api(path, options = {}) {
  const response = await fetch(`${BASE}${path}`, {
    headers: { 'Content-Type': 'application/json', ...(options.headers || {}) },
    ...options,
  })

  return response.json()
}
```

まず、データを受け取るためにURLを設定する
fetchを使ってpathを設定しながら、色々なデータを取得できる

```
import { useEffect, useState } from 'react'
import { api } from '../api/client'

export default function Setting() {...
```

```
import Setting from './pages/Setting'
```

```
<Route path="/setting" element={<Setting/>}/>
```

Setting.jsxを生成して、コンポーネントに使う

そして、App.jsxにRouteを設定する

```
const [sensors, setSensors] = useState([]);
```

```
useEffect(() => {
  api('/sensors')
    .then(data => {
      setSensors(data)
    })
}, [])
```

```
<table>
  <thead>
    <tr>
      <td>コード</td>
      <td>名称</td>
      <td>建物</td>
      <td>種別</td>
      <td>現在値</td>
      <td>状態</td>
      <td>操作</td>
    </tr>
  </thead>
  <tbody>
    {
      sensors.map(sensor => (
        <tr key={sensor.id}>
          <td>{sensor.sensorCode}</td>
          <td>{sensor.sensorName}</td>
          <td>{sensor.buildingName}</td>
          <td>{sensor.sensorType}</td>
          <td>{sensor.latestValue}</td>
          <td>{sensor.active ? "稼働" : "停止"}</td>
          <td><button>ボタン</button></td>
        </tr>
      ))
    }
  </tbody>
</table>
</>
```

データを照会するため、ReactのuseStateを生成した後、useEffectを使ってデータを一度だけ取得する
取得したデータは宣言したuseStateに保存する
この多量のデータはmapを使って表示する

TMP-001	1F Office Temperature	Tokyo HQ	TEMPERATURE	24.8	稼働	ボタン
HUM-001	1F Office Humidity	Tokyo HQ	HUMIDITY	45.2	稼働	ボタン
TMP-101	Production Area Temperature	Osaka Factory	TEMPERATURE	27.3	稼働	ボタン

登録

```
const [showForm, setShowForm] = useState(false);
const [form, setForm] = useState({
  buildingId: '',
  sensorCode: '',
  sensorName: '',
  sensorType: '',
  unit: '',
  minThreshold: '',
  maxThreshold: '',
  active: true
});
```

登録するためにFormに入力することをuseStateを使って設定する

```
<h1>センサー管理</h1>
<button onClick={() => setShowForm(true)}>新規登録</button>
{
  showForm && (
    <div>
      <form onSubmit={handleCreate}>

        <input
          placeholder='建物ID'
          value={form.buildingId}
          onChange={(e) =>
```

```

        setForm({
            ...form, buildingId :
e.target.value
        })
    }
    />

<input
    placeholder='コード'
    value={form.sensorCode}
    onChange={(e) =>
        setForm({
            ...form, sensorCode :
e.target.value
        })
    }
    />

<input
    placeholder='名称'
    value={form.sensorName}
    onChange={(e) =>
        setForm({
            ...form, sensorName :
e.target.value
        })
    }
    />

<input
    placeholder='種別'
    value={form.sensorType}
    onChange={(e) =>
        setForm({
            ...form, sensorType :
e.target.value
        })
    }
    />

```

```

        />

        <select
            value={form.unit}
            onChange={(e) =>{
                setForm({
                    ...form, unit: e.target.val
ue
                })
            }}
        >
            <option value="">選択</option>
            <option value="°C">°C</option>
            <option value="%">%</option>
            <option value="ppm">ppm</option>
        </select>

        <input
            placeholder='minThreshold'
            value={form.minThreshold}
            onChange={(e) =>
                setForm({
                    ...form, minThreshold :
e.target.value
                })
            }
        />

        <input
            placeholder='maxThreshold'
            value={form.maxThreshold}
            onChange={(e) =>
                setForm({
                    ...form, maxThreshold :
e.target.value
                })
            }
        />

```

```

    }
    />
    <button type="submit">登録</button>
  </form>
</div>
)
}

```

登録ボタンを押すと、隠れていたformが出るようにするので、showFormをuseStateにして、trueの場合表現するようにする

```

async function handleCreate(e) {
  e.preventDefault();

  await api('/sensors', {
    method : 'POST',
    body : JSON.stringify(form)
  })

  const data = await api('/sensors')
  setSensors(data)

  setForm({
    buildingId: '',
    sensorCode: '',
    sensorName: '',
    sensorType: '',
    unit: '',
    minThreshold: '',
    maxThreshold: ''
  })

  setShowForm(false)
}

```

handleCreateを見ると、まずデータを送信して、SpringBootに送る
その後、登録したデータをまとめて、全体データを取得する

後、formの内容は初期化する

```
@PostMapping
public Sensor create(@Valid @RequestBody SensorRequest req) {
    return service.create(req);
}
```

このように送信したデータを、Controller → Service → MyBatisを通じてDBに保存する

The screenshot shows a web form for registration. At the top left is a button labeled '新規登録' (New Registration). Below it is a table with seven columns: '建物ID' (Building ID), 'コード' (Code), '名称' (Name), '種別' (Type), '選択' (Selection) with a dropdown arrow, 'minThreshold', and 'maxThreshold'. At the bottom left of the table is a button labeled '登録' (Register).

登録画面はこんなに簡単に作る

修正

```
const [editId, setEditId] = useState(null);
```

```
<td>
  <button onClick={() => handleEdit(sensor)}>
    編集
  </button>
</td>
```

編集のため、テーブル内容ごとに編集を付ける

```
function handleEdit(sensor) {
  setEditId(sensor.id)

  setForm({
    buildingId: sensor.buildingId,
    sensorCode: sensor.sensorCode,
    sensorName: sensor.sensorName,
    sensorType: sensor.sensorType,
    unit: sensor.unit,
    minThreshold: sensor.minThreshold,
    maxThreshold: sensor.maxThreshold,
    active: sensor.active
  })

  setShowForm(true)
}
```

そして、sensor値を受け取った後、登録画面に使ったformを再利用する

```
async function handleSubmit(e) {

  e.preventDefault();

  if(editId == null) {
    await api('/sensors', {
      method : 'POST',
      body : JSON.stringify(form)
    })
  } else {
    await api(`/sensors/${editId}`, {
      method : 'PUT',
      body : JSON.stringify(form)
    })
  }
}
```

そのためにhandleCreateをhandleSubmitに変更
 handleEdit()でeditIdの値を設定した後、if文を使って修正と登録を選択する
 そして、SpringBootに接近する


```
<form onSubmit={handleSubmit}>
```

```
<button type="submit">
  {
    editId ? "編集" : "登録"
  }
</button>
```

formはhandleSubmitに
editIdがある場合、ボタンを編集にする

コード	名称
TMP-003	1F Office Temperature

2F Office Temperature	TEMPERATURE	°C	24	35	編集
-----------------------	-------------	----	----	----	----

コード	名称
TMP-003	2F Office Temperature

このように編集できる